System design

使用 Windows 作為主機系統，並使用 WSL2 Ubuntu 作為 Linux kernel 開發環境，在project中維護了兩份kernel source tree linux-6.6.30-standard2 和 linux-6.6.30-modified 以進行Task 4 的測試，並在modified kernel 新增了兩個要求的system call。
檔案系統部分使用 BusyBox 建立最小 root filesystem 。最後透過 QEMU 模擬 x86_64 架構，使用自行編譯的 bzImage 與 initramfs 啟動 kernel。

implementation details

(1)Task1

1. 在WSL2中安裝必要的工具
2. 載 Linux 6.6.30 kernel source，使用 make defconfig 產生預設設定，並透過 make -j$(nproc) 編譯 kernel。編譯完成後，kernel image 產生於 arch/x86/boot/bzImage。
3. 使用 BusyBox 建立最小 root filesystem
4. 用 cpio 與 gzip 將 rootfs 打包成 initramfs.cpio.gz。最後使用 QEMU 指定 bzImage 與 initramfs.cpio.gz 啟動 kernel

(2)Task2

1. 在linux-6.6.30-modified/kernel/sys.c 檔案中加入mycall的實作
2. 在linux-6.6.30-modified/include/linux/syscalls.h 加入prototype:
   asmlinkage long sys_mycall(int x);
3. 在linux-6.6.30-modified/arch/x86/entry/syscalls/syscall_64.tbl 加入新的syscall entry:
   454 common mycall sys_mycall
4. 寫測試程式 test_mycall.c
5. 使用static linking編譯程式
   gcc -static test_mycall.c -o test_mycall
6. 複製到 rootfs
7. 重新打包 initramfs

(3)Task3 (Choose direction3)

1. 為了讓 `kernel-space` 與 `user-space` 使用相同的資料格式，新增一個 `UAPI header: linux-6.6.30-modified/include/uapi/linux/proc_info.h`
2. 在linux-6.6.30-modified/kernel/sys.c 檔案中加入get_proc_info的實作
3. linux-6.6.30-modified/include/linux/syscalls.h 中加入get_proc_info的宣告
4. 在linux-6.6.30-modified/arch/x86/entry/syscalls/syscall_64.tbl 加入新的syscall entry:
   455 common get_proc_info sys_get_proc_info
5. 寫測試程式 test_get_proc_info.c
6. 使用static linking編譯程式
   gcc -static test_get_proc_info.c -o test_get_proc_info
7. 複製到 rootfs
8. 重新打包 initramfs

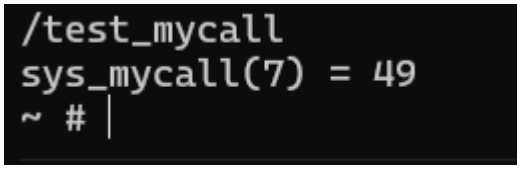
(4)Task4

1. 建立名為 benchmark.c 的檔案
2. 使用static linking編譯程式
3. 複製到 rootfs
4. 重新打包 initramfs

testing methodology

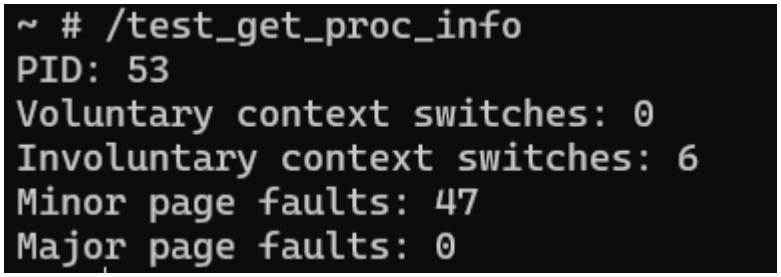
(1)Task2

在 QEMU 中執行：/test_mycall



(2)Task3

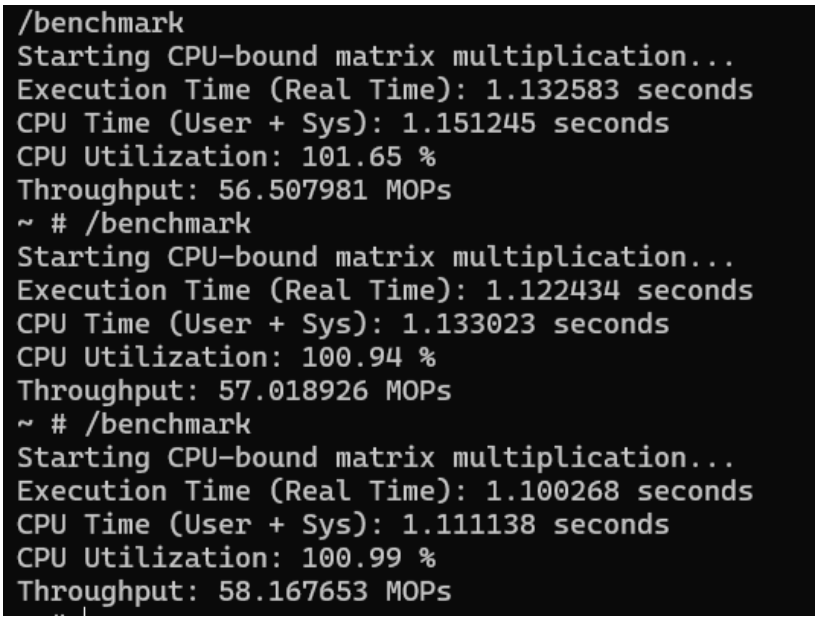
在 QEMU 中執行：/test_get_proc_info



(3)Task4

在 QEMU 中執行:/benchmark

standard:

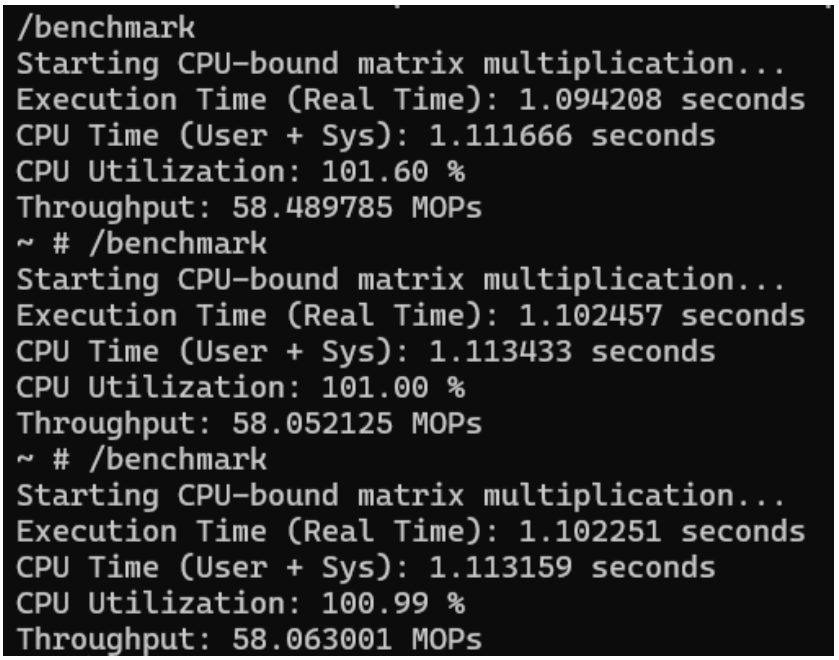


Average Execution Time: 1.118s

Average CPU Utilization: 101.19%

Average Throughput: 57.232 MOPs

modified:



Average Execution Time: 1.099s

Average CPU Utilization: 101.20%

Average Throughput: 58.202 MOPs

performance analysis

從測試結果來看，standard kernel 和 modified kernel 的效能差異很小。modified kernel 的平均執行時間略低，throughput 略高，但差距不大。

這個結果符合預期，因為我們的 kernel 修改主要是新增 sys_mycall 和 sys_get_proc_info 兩個 system call。benchmark.c 執行時並不會反覆呼叫這兩個新增的 system call，因此對 CPU-bound benchmark 的執行效能幾乎沒有影響。

兩者之間的小幅差異可能來自 QEMU 執行環境、CPU scheduling、系統背景負載或量測誤差，而不是 kernel 修改本身造成的明顯效能變化。

AI-assisted development process

AI主要輔助我理解 System call的新增流程，例如新增檔案的位置和它們的功能;另外因為我對終端機的指令不太熟悉，AI在這方面也給我許多幫助